System Requirements Specification (SRS)

1.1 Purpose & Scope

The purpose of this project is to develop a lightweight, secure Peer-to-Peer (P2P) file transfer application for Local Area Networks (LANs). It aims to replace insecure traditional FTP systems by implementing a transient Client-Server architecture over TCP sockets. To secure the files against Man-in-the-Middle (MITM) attacks or packet sniffing, it utilizes a **Hybrid Cryptography approach (AES + ECC)** inspired by the base paper.

1.2 Functional Requirements

- **Connection Management:** The system must allow one node to act as a Server (listener) and another as a Client (initiator) using IPv4 and port binding.
- **Authentication & Key Exchange:** Before file transmission, the peers must authenticate and securely exchange a one-time cryptographic session key.
- **File Selection & Reading:** The user must be able to browse and select a file of any format (.txt, .pdf, .mp4, etc.) from their local storage.
- **Encryption/Decryption:** The system must encrypt the file using a symmetric algorithm (AES) and encrypt the AES key using an asymmetric algorithm (ECC).
- **Data Transmission:** The system must chunk the encrypted file and send it over the LAN using a TCP socket stream.
- **Integrity Verification:** The receiver must calculate a cryptographic hash (e.g., SHA-256) of the decrypted file to ensure no data corruption occurred.

1.3 Non-Functional Requirements

- **Security:** High. Data must be unreadable to anyone sniffing the LAN packets (e.g., using Wireshark).
- **Performance:** Encryption and decryption must not cause severe bottlenecks. The LAN transfer speed should ideally bottleneck the process, not the CPU.
- **Reliability:** TCP (Transmission Control Protocol) must be used to guarantee packet delivery and order.

1.4 Tech Stack Recommendations

- **Programming Language:** Python (Highly recommended for network and cryptography libraries).
- **Networking Library:** Built-in `socket` library.
- **Cryptography Library:** `PyCryptodome` (for AES) and `tinyec` or `cryptography` (for ECC/Key Exchange).
- **GUI (Optional):** `Tkinter` or `PyQt5` for an easy-to-use user interface.

Theoretical Knowledge Foundation

To implement and document this successfully, you need to understand four core pillars:

A. Client-Server vs. P2P in LAN

While the overall architecture is Peer-to-Peer (meaning no centralized cloud server is involved), the actual transaction uses a **transient Client-Server model**.

- **The Receiver (Server):** Opens a port on their local IP (e.g., `192.168.1.5:5001`) and listens for incoming connections.
- **The Sender (Client):** Connects to the Receiver's IP and port to initiate the handshake and file push.

B. Socket Programming (TCP)

Sockets are the endpoints of a two-way communication link. You must use **TCP (SOCK_STREAM)** rather than UDP (SOCK_DGRAM) because TCP guarantees that no file packets are lost or delivered out of order during transmission.

C. The Base Paper Concept: Hybrid Cryptography (AES + ECC)

Your base paper solves the security dilemma of cloud storage by mixing two algorithms. You will adapt this to LAN networks:

1. **Symmetric Encryption (AES-256):** Very fast, great for bulk data (the actual file). However, it requires both peers to have the *same* secret key. If you send the key over the LAN, a hacker can steal it.
2. **Asymmetric Encryption (ECC - Elliptic Curve Cryptography):** Uses a Public and Private key pair. It is highly secure and requires smaller key sizes than RSA, but it is too slow to encrypt large files.
3. **The Hybrid Solution:**
    - Peer A generates a random AES Key.
    - Peer A encrypts the *file* using the AES Key.
    - Peer A encrypts the *AES Key* using Peer B's ECC Public Key.
    - Peer A sends both the encrypted file and encrypted key to Peer B.

    - Peer B decrypts the key using their ECC Private Key, then uses the AES key to decrypt the file.

D. File Transfer Protocol (FTP) Mechanisms

While you aren't using the official Port 21 FTP protocol, you are mimicking its mechanism:

- **Control Channel:** Used first to send metadata (File name, File size, Hash, Encrypted AES key).
- **Data Channel:** Used second to stream the actual encrypted binary chunks of the file.

Step-by-Step Implementation Strategy

Step 1: Environment & Network Setup

- Create a virtual environment in Python.
- Write a basic script to print the machine's local IP address (`socket.gethostbyname(socket.gethostname())`).
- Create two scripts: `sender.py` and `receiver.py`.

Step 2: Establish the Socket Connection

- **Receiver:** Use `socket.bind((IP, PORT))` and `socket.listen()`. Wait for a connection using `socket.accept()`.
- **Sender:** Use `socket.connect((RECEIVER_IP, PORT))`.
- *Test:* Send a simple string like "Hello" to ensure the LAN connection works without firewall blockage.

Step 3: Implement ECC Key Exchange (The Authentication Phase)

- When the connection starts, the **Receiver** generates an ECC Public/Private key pair.
- The Receiver sends its *Public Key* over the socket to the Sender.
- The Sender generates a random 256-bit AES session key.
- The Sender encrypts this AES session key using the Receiver's ECC Public key, and sends it back.
- The Receiver decrypts it. *Now both computers securely share the same AES key.*

Step 4: File Reading, Encryption, and Hashing

- **Sender:** Open the target file in binary read mode (`rb`).
- Calculate the SHA-256 hash of the original file (to send as metadata for integrity checking).
- Initialize the AES Cipher in GCM or CBC mode using the shared session key.
- Encrypt the file data. (For large files, read and encrypt in chunks of 4KB or 8KB).

Step 5: Protocol Formatting (Metadata + Data)

Don't just stream raw data. Send a metadata header first.

- **Sender sends:** `FILENAME | FILESIZE | SHA-256 HASH | <END_OF_HEADER>`
- Wait for Receiver to acknowledge the header.
- Stream the encrypted file chunks over the socket.

Step 6: Decryption & Reassembly

- **Receiver:** Reads the header to know exactly how many bytes to expect (preventing infinite hanging loops).
- Receives the encrypted chunks and passes them through the AES decryptor.
- Writes the decrypted chunks to a new file on the disk (`wb` mode).

Step 7: Integrity Check

- Once the file is saved, the Receiver calculates the SHA-256 hash of the newly saved file.
- Compare it to the hash received in the header. If they match, the transfer is 100% successful and untampered.

Documentation & Research Paper Strategy

To write a strong research paper, you must clearly highlight how your project builds upon the base paper but targets a different networking environment.

Recommended Paper Structure

1. Abstract:

Summarize the problem (insecure LAN sharing), the methodology (P2P socket programming with Hybrid AES-ECC), and the result (secure, lightweight, fast file transfer without internet dependency).

2. Introduction:

Discuss the widespread use of LANs in offices and schools. Explain the vulnerabilities of sending files over unencrypted USB drives or standard FTP. Introduce the need for end-to-end cryptography.

3. Literature Review (Where you cite the base paper):

- Critique standard file sharing (SMB, standard FTP) for lack of built-in encryption.
- *Compare with existing solutions:* Mention cloud services (Google Drive, Dropbox) which provide security but require high internet bandwidth, external trust, and incur latency.
- *Cite your base paper:* Note how MDPI 10(21):2673 proposed a Hybrid AES-ECC model to secure data *on the cloud*. Point out that your research adapts this exact powerful cryptographic hybrid model into a **server-less, offline, localized P2P architecture**.

4. Proposed System Architecture:

Create block diagrams showing:

- The system topology (Node A to Node B via Switch/Router).
- The Cryptographic flow (How ECC public keys are exchanged, how the AES key is encrypted/decrypted, and how the file data flows).

5. Methodology & Implementation (The "How-to"):

Discuss TCP socket states, the metadata header structure, the chunking mechanism for large files (to prevent RAM overload), and the mathematical benefits of using ECC over RSA (smaller key size means less network overhead during the handshake).

6. Experimental Results & Performance Analysis:

Include a table comparing your system against existing solutions:

- Transfer time for a 10MB, 100MB, and 1GB file.
- Time taken for the cryptographic handshake (usually under 0.5 seconds).
- *Security analysis:* What happens if a bad actor runs Wireshark on the network? (Answer: They only see encrypted TCP payloads and an ECC public key, they cannot derive the file).

7. Conclusion & Future Scope:

Summarize the success of the hybrid localized model. Future scope could include adding Multi-cast (sending securely to multiple PCs in the LAN at once) or integrating an intrusion detection alert system.

How it differs from Existing Solutions (Your core argument):

1. **Vs. Standard FTP:** Standard FTP sends data in plaintext. Your system ensures zero plaintext transmission.
2. **Vs. Cloud Solutions (Base Paper context):** The base paper relies on uploading data to a central, remote third-party cloud. Your solution cuts out the middleman, offering **zero-trust localized sharing**. It works even if the internet is down, as long as the local router/switch is active.
3. **Vs. Pure AES Transfer:** A pure AES system requires users to manually share a password (out-of-band). By integrating ECC (Asymmetric), your system achieves automated, mathematical key exchange over an insecure channel.